

3. Programació amb arrays i objectes

En aquesta lliçó explorarem com JavaScript permet treballar amb dades estructurades i comportaments associats mitjançant objectes. Aprendre a crear i gestionar **arrays** i **objectes** definits pel programador, i introduïrem els principis bàsics de la programació orientada a objectes (POO), com ara classes, propietats i mètodes. Finalment, veurem alguns exemples pràctics que mostren com aplicar aquests conceptes per escriure codi més modular, reutilitzable i fàcil de mantenir.

3.1 Arrays. Objectes definits pel programador

En aquest apartat veurem com JavaScript ens permet organitzar i gestionar dades de manera eficient mitjançant arrays i objectes. Els arrays són col·leccions ordenades d'elements, ideals per emmagatzemar llistes o conjunts de valors, mentre que els objectes ens permeten agrupar informació relacionada en una mateixa estructura amb claus i valors. A més, combinant aquests elements, podem crear estructures de dades complexes adaptades a les necessitats del nostre programa.

3.1.1 Arrays com estructures de dades

Un **array** és una estructura que ens permet **emmagatzemar diversos valors en una sola variable**, de forma **ordenada**. És molt útil quan volem treballar amb **llistes de dades**, com noms, números, objectes, etc.

Els arrays en JavaScript poden contenir **qualsevol mena de dada**, fins i tot barrejats (nombres, *strings*, booleans, objectes, altres arrays...).

Creació d'un array

A continuació, veurem alguns exemples de com crear arrays.

```
1 let buida = []; // Array buit
2 let numeros = [1, 2, 3, 4]; // Array numèric
3 let noms = ["Anna", "Pau", "Joan"]; // Array d'Strings
4 let mesclat = [1, "hola", true, [10, 20]]; // Array mixt
```

També es pot fer amb el constructor:

```
1 let colors = new Array("vermell", "blau", "verd");
```

Tot i això, la forma recomanada és la de claudàtors ([]), per ser més clara i segura.

Accés als elements

Els elements es compten des de l'índex zero (0 és el primer element).

```
1 let numeros = [10, 20, 30];
2 console.log(numeros[0]); // 10
3 console.log(numeros[2]); // 30
```

Longitud de l'array

Cada array té una propietat `.length` que indica quants elements conté:

```
1 numeros.length; // 3
```

Aquesta propietat també es pot usar per recórrer l'array amb un bucle:

```
1 for (let i = 0; i < numeros.length; i++) {
2   console.log(numeros[i]);
3 }
```

Mètodes útils dels arrays

Els arrays disposen de molts **mètodes integrats** per facilitar les operacions habituals. A continuació veurem un llistat dels més usuals:

TAULA 3.1. Mètodes útils dels arrays

Mètode	Què fa
<code>push()</code>	Afegeix un element al final
<code>pop()</code>	Elimina i retorna l'últim element
<code>unshift()</code>	Afegeix un element al principi
<code>shift()</code>	Elimina i retorna el primer element
<code>join(separador)</code>	Uneix tots els elements en un <i>string</i> , separats per separador. El separador per defecte és la coma ,
<code>concat()</code>	Uneix dos arrays en un de sol
<code>slice(inici, fi)</code>	Retorna una part de l'array (no modifica l'original)
<code>splice()</code>	Afegeix/elimina elements de l'array (sí que modifica l'original)
<code>indexOf(elem)</code>	Retorna la posició d'un element (o -1 si no hi és)
<code>includes(elem)</code>	Indica si un element es troba dins l'array

Anem a veure alguns exemples del mètodes anteriors:

```
1 let fruites = ["poma", "plàtan", "kiwi"];
2
3 fruites.push("maduixa");           // ["poma", "plàtan", "kiwi", "
   maduixa"]
```

```
4 fruites.shift();           // ["plàtan", "kiwi", "maduixa"]
5 fruites.includes("kiwi");   // true
6 fruites.indexOf("plàtan");  // 0 (Ha canviat després d'
    aplicar el mètode shift)
```

Iterar amb ".forEach()"

El mètode `.forEach()` ens permet **recórrer tots els elements d'un array** i executar una funció per a cadascun d'ells. És una alternativa moderna i més llegible que els bucles `for`.

No retorna cap nou array: simplement **executa una acció per cada element**.

```
1 let animals = ["gat", "gos", "ocell"];
2
3 animals.forEach(function(animal, index) {
4     console.log(index + ": " + animal);
5 });
```

També es pot fer amb funcions fletxa:

```
1 animals.forEach((animal, i) => console.log(i + ": " + animal));
```

Transformar arrays: ".map()" i ".filter()"

Els mètodes `.map()` i `.filter()` ens permeten crear un nou array a partir de l'original:

- `map()` transforma cada element i retorna un array nou amb els resultats.
- `filter()` selecciona només els elements que compleixen una condició.

Aquests mètodes no modifiquen l'array original.

```
1 let nums = [1, 2, 3, 4];
2
3 // Multiplica per 2 cada element
4 let dobles = nums.map(n => n * 2); // [2, 4, 6, 8]
5
6 // Filtra els nombres parells
7 let parells = nums.filter(n => n % 2 === 0); // [2, 4]
```

3.1.2 Objectes com estructures de dades

A JavaScript, un **objecte** és una estructura que ens permet **emmagatzemar dades en forma de clau-valor**. És una eina fonamental per representar entitats més complexes i flexibles que un array, com ara una persona, un cotxe, un usuari, etc.

Mentre que els arrays organitzen les dades per **posició numèrica (índexs)**, els objectes ho fan per **claus amb nom (strings)**.

Creació d'un objecte

A continuació, es mostra com crear un **objecte literal**. Un objecte literal en JavaScript és una forma directa i senzilla de crear un objecte amb *claus* (propietats) i *valors*.

```
1 let persona = {  
2   nom: "Anna",  
3   edat: 28,  
4   viu: true  
5 };
```

Cada propietat té una *clau* (ex. “nom”) i un *valor* associat (ex. “Anna”).

Accés a propietats

Podem accedir a les propietats amb notació de punt o notació de claudàtors:

```
1 // Punt (dot notation)  
2 console.log(persona.nom); // "Anna"  
3  
4 // Claudàtors (bracket notation)  
5 console.log(persona["edat"]); // 28
```

Notació amb claudàtors

La notació amb claudàtors és útil quan la clau és una variable o conté espais:

```
1 let clau = "viu";  
2 console.log(persona[clau]); // true
```

Modificar i afegir propietats

Els objectes en JavaScript són **dinàmics**, la qual cosa vol dir que podem afegir noves propietats o modificar-ne les existents en qualsevol moment, fins i tot després d’haver estat creats.

Aquesta flexibilitat fa que els objectes siguin molt útils com a contenidors d’informació estructurada.

```
1 persona.edat = 29; // Modifiquem  
2 persona.professio = "dissenyadora"; // Afegim nova propietat
```

Mètodes dins d'un objecte literal

A més de propietats amb valors com números o textos, un objecte literal també pot incloure **funcions**. Quan una funció forma part d’un objecte, l’anomenem **mètode**.

Aquests mètodes permeten definir accions o comportaments relacionats amb l'objecte.

```
1 let persona = {
2   nom: "Anna",
3   edat: 28,
4   viu: true,
5   saludar() {
6     console.log('Hola! Soc ${this.nom}.');
7   }
8 };
9
10 persona.saludar(); // "Hola! Soc Anna."
```

Els mètodes poden accedir a les propietats de l'objecte mitjançant la paraula clau `this`, que fa referència a l'objecte mateix.

Eliminar propietats

La paraula clau `delete` serveix per **eliminar propietats d'un objecte**. Quan s'utilitza, **esborra la clau i el valor** associat, de manera que ja no existeixen dins de l'objecte.

```
1 delete persona.viu;
```

També podem eliminar **mètodes** (funcions dins d'un objecte), ja que internament són propietats com qualsevol altra.

```
1 let persona = {
2   nom: "Anna",
3   edat: 29,
4   saludar() {
5     console.log('Hola, soc ${this.nom}');
6   }
7 };
8
9 persona.saludar(); // "Hola, soc Anna"
10 delete persona.saludar; // Eliminem el mètode
11 console.log(persona.saludar); // undefined
```

Després d'eliminar-lo, el mètode ja no està disponible dins de l'objecte.

Recórrer les propietats amb "for...in"

Quan treballem amb objectes, sovint volem **recórrer totes les seves propietats** per llegir o processar els valors que contenen. En JavaScript, això es pot fer fàcilment amb el bucle especial **for...in**.

Aquest bucle ens permet **iterar per totes les claus (propietats enumerables)** d'un objecte.

```
1 for (let clau in persona) {
2   console.log(clau + ": " + persona[clau]);
}
```

```
3 }  
4 // nom: Anna  
5 // edat: 29  
6 // professio: dissenyadora
```

Els objectes són especialment útils quan necessitem **representar entitats complexes** que tenen diverses característiques o propietats. Per exemple, si volem descriure un usuari, un llibre o un producte, cadascun d'aquests elements tindrà múltiples dades associades, com el nom, l'edat, el preu o la categoria. En aquests casos, un array no seria suficient, ja que només ens permet emmagatzemar valors de manera ordenada, però **sense significat semàntic clar**.

A diferència dels arrays, que organitzen la informació per posició (índex), els objectes ens permeten accedir a cada dada mitjançant **una clau descriptiva**, com nom, edat o preu. Això fa que el codi sigui més **llegible i entenedor**, sobretot quan treballem amb dades estructurades.

En resum, utilitzarem objectes sempre que vulguem **estructurar informació amb sentit** (estructures de dades), i no només llistes de valors. Són una eina fonamental per construir programes clars, escalables i fàcils de mantenir.

3.1.3 Estructures de dades complexes

Quan treballem amb dades reals en una aplicació, sovint no n'hi ha prou amb estructures simples com arrays o objectes independents. En aquests casos, JavaScript ens permet **combinar arrays i objectes** per construir **estructures de dades complexes**, molt més potents i representatives del món real.

Arrays que contenen objectes

Una de les combinacions més habituals és tenir un **array d'objectes**. Aquesta estructura és ideal quan volem representar una col·lecció d'elements que comparteixen una mateixa estructura, com ara una llista de llibres, d'usuaris o de productes. Cada objecte de l'array representa un element amb les seves pròpies propietats.

Per exemple, imaginem que volem representar un catàleg de llibres:

```
1 let llibres = [  
2   { titol: "1984", autor: "George Orwell", any: 1949 },  
3   { titol: "Crim i càstig", autor: "Fiódor Dostoievski", any:  
4     1866 },  
5   { titol: "L'ombra del vent", autor: "Carlos Ruiz Zafón", any:  
     2001 }  
];
```

En aquest cas, l'array `llibres` conté tres objectes, cadascun amb informació rellevant sobre un llibre. Això ens permet, per exemple, recórrer tota la llista

amb un bucle i mostrar només els títols o buscar l'autor d'un llibre concret.

Aquesta estructura és especialment útil quan gestionem dades provinents d'una API o una base de dades, ja que aquestes normalment retornen col·leccions d'elements representats com objectes dins d'un array.

Objectes amb propietats que són arrays

També és molt comú el cas invers: **objectes que contenen arrays com a valors d'algunes propietats**. Això ens permet representar característiques que poden tenir **múltiples valors associats**, com per exemple les aficions d'una persona, les notes d'un alumne, o les etiquetes d'un producte.

Observem aquest exemple:

```
1 let usuari = {  
2   nom: "Marta",  
3   edat: 22,  
4   aficions: ["lectura", "escalada", "viatjar"]  
5 };
```

Aquí l'objecte usuari conté una propietat aficions que és un array. Això ens permet treballar de manera eficient amb múltiples valors relacionats amb un sol camp, mantenint una estructura lògica i clara.

Podem accedir-hi i treballar-hi fàcilment:

```
1 console.log(usuari.aficions[1]); // "escalada"  
2 usuari.aficions.push("cinema");
```

Aquest tipus de combinacions ens permeten **modelar la realitat** dins del nostre programa. A diferència d'un full de càlcul o una simple llista de valors, aquí podem representar relacions complexes, jerarquies i col·leccions d'informació estructurada.

És gràcies a aquestes estructures que podem construir funcionalitats com:

- Llistes de tasques amb subtasques
- Perfils d'usuari amb múltiples preferències
- Carretons de compra amb productes i quantitats
- Aplicacions de calendari amb esdeveniments per dies

Dominar les estructures de dades complexes ens permet construir aplicacions **més potents i realistes**. JavaScript ens dona la flexibilitat de **combinar arrays i objectes lliurement**, cosa que és fonamental per treballar amb dades de forma organitzada, neta i escalable.